

A Low Power and High-Speed 32-bit Adaptive ALU Architecture for FPGA-Based Computing

1st Nguyen Tuan Minh, 2nd Nguyen Duc Minh, 3rd Hoang Tuan Hiep, and Mentor Dang Van Hieu
FPT University, Ha Noi, Viet Nam
{minh26ht, d.minh71205, hhiep1905}@gmail.com, and hieudv2@fe.edu.vn

Abstract

Base ALU architectures process every operand pair along the same fixed datapath, applying full-width carry propagation and activating all functional units regardless of the exact operand complexity. Such uniform computing causes unnecessary switching activity and wastes energy when operand magnitudes are often small or sparse - a prevalent case in sensor and low-power embedded applications. This paper introduces an adaptive ALU that introduces data-dependent adaptivity into the arithmetic path against a conventional shared-resource ALU baseline (from the lowRISC Ibex core), which optimizes for area via hardware reuse.

This design, therefore, involves three cooperating mechanisms not found in the baseline architecture. First, a complexity classifier checks the MSB of both operands every cycle, qualifying them and gating the adaptive path only when such qualification is met. Second, when enabled via the power-save control signal and when operands are qualified as simple, the segmented carry-breaking adder decomposes the addition into lower and upper bit segments that can operate independently by suppressing carry propagation across their boundary, thus shortening the effective carry chain and reducing switching activity at the cost of bounded approximation. This approximation is protected by a same-cycle error feedback signal indicating any case in which the suppressed carry would have altered the correct result, thus maintaining correctness visibility rather than silently accepting error. Third, the multiplier datapath uses operand isolation by zeroing the inputs whenever the active opcode is not multiplied, eliminating idling switching in this high-fanout submodule.

Index Terms

Adaptive ALU, Low-power Design, Operand Gating, SAIF Simulation, RISC-V, ESP32, FPGA Implementation, Hardware-in-the-Loop.

I. INTRODUCTION

In the era of the IoT and Edge AI, microcontrollers are increasingly deployed in power-constrained environments. Processor cores compatible with the RISC-V RV32IMC architecture, such as those inside the ESP32-C series [1], represent a major hardware baseline for smart edge devices. Inside these processors, the Arithmetic Logic Unit (ALU) remains a primary contributor to the total dynamic power consumption because continuous switching activities during instruction execution [2].

Traditional ALUs are statically optimized for the worst-case scenario, activating entire internal datapaths (e.g., full 32-bit carry propagation and complex multiplication trees) regardless of the actual data magnitude [2]. However, typical IoT workloads often process small data, small loop counters, or low precision sensor outputs, leaving the upper bits of ALU operands mostly inactive (fill-zeros), consistent with prior observations that ALU power consumption is strongly data-dependent [3].

While system-level techniques like Dynamic Voltage and Frequency Scaling (DVFS) improve broad power savings [4], they cannot adapt to clock-cycle-level variations of input data. Approximate computing adders trade arithmetic precision for lower propagation delays and switching activity [5], [6], but their lack of determinism prevents their usage in critical, general-purpose software pipelines.

In this work, the proposed adaptive ALU is benchmarked against a conventional shared-resource ALU baseline derived from the lowRISC Ibex core [7], which optimizes for area via hardware reuse rather than data-dependent adaptivity.

To bridge this gap, this paper presents a 32-bit Adaptive ALU featuring:

- An autonomous Complexity Estimator that dynamically identifies operand data-width.
- A Segmented Carry-Breaking Adder that truncates the critical carry chain during low-complexity arithmetic operations.
- An instantaneous Combinational Error-Recovery (0-cycle latency) circuit that resolves carry-out errors without stalling or flushing the instruction pipeline.
- An Operand Isolation (Gating) mechanism at the inputs of the 32-bit multiplier to eliminate redundant switching power.
- A dynamic hardware-in-the-loop configuration utilizing a physical slide switch to enable/disable power-saving features and an onboard LED to visually signal real-time carry-error correction.
- A rigorous SAIF-driven simulation methodology using realistic switching activity to validate the exact dynamic power savings on FPGA hardware.

II. RELATED WORK

Recent literature has extensively explored micro-architectural energy optimization techniques for Arithmetic Logic Units (ALUs). To reduce dynamic power consumption, conventional methods frequently employ clock gating or power gating.

However, implementing clock gating on FPGA fabrics often introduces severe clock skew and timing hazards, while power gating suffers from considerable wake-up latency and area overhead. To address these limitations, operand isolation has emerged as a non-intrusive alternative. By forcing both input operands to zero when multiplier block or other complexity modules is not activated, it avoids waste dynamic power when bits toggle continuously.

In parallel, approximate computing has been widely adopted to accelerate arithmetic datapaths, particularly through the use of Segmented Carry Prediction Adders (SCPA) [8]. These architectures reduce the critical path delay by full-width carry propagation chain into independent smaller segments. Nevertheless, conventional approximate adders operate under fixed, static configurations, leading to unrecoverable computational errors when processing high-complexity workloads. To improve reliability, dynamic adaptation schemes have been proposed; however, current error-correction methods often require pausing the pipeline or taking multiple clock cycles to fix errors, which severely slows down the processing speed when errors occur frequently.

This paper directly addresses these challenges by introducing a low-power, high-speed 32-bit adaptive ALU architecture. Unlike base designs, the proposed architecture incorporates a hardware-efficient complexity estimator to dynamically toggle between exact and approximate execution modes. Furthermore, a 0-cycle combinational error-detection and recovery datapath is introduced to instantly rectify critical carry-overflow errors without incurring pipeline stalls, achieving a robust balance between energy efficiency and computational accuracy.

III. PROPOSED 32-BIT ADAPTIVE ALU ARCHITECTURE

The overall hardware organization of the proposed design consists of an adaptive execution core controlled by a dedicated pre-processing layer, as depicted in the Verilog-2001 architecture.

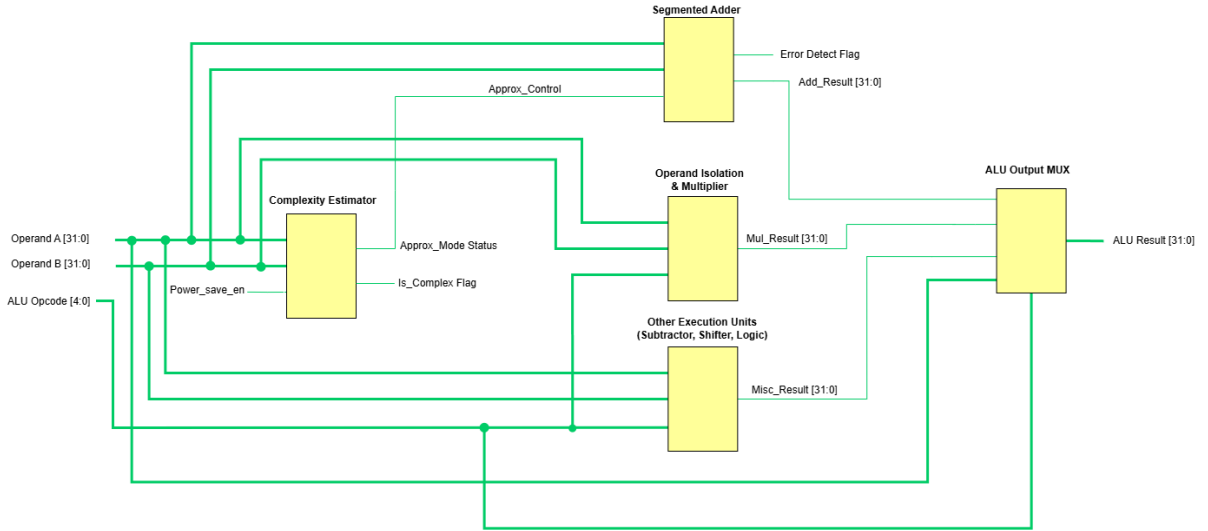


Fig. 1. Hardware block diagram of the proposed 32-bit Adaptive ALU with Complexity Estimator.

A. Complexity Estimator and Mode Controller

The Complexity Estimator evaluates the upper 16 bits - the Most Significant Bits (MSBs) of both 32-bit operands, A and B , to determine if the operation can be simplified. The complexity status flag $add_is_complex_w$ is formulated as:

$$add_is_complex_w = \left(\bigvee_{i=16}^{31} A[i] \right) \vee \left(\bigvee_{i=16}^{31} B[i] \right) \quad (1)$$

In the hardware implementation, this formulation is mapped using the unary reduction OR operation. Instead of comparing the upper 16 bits to a 16-bit zero value ($alu_a_i[31:16] = 16'h0$), the adaptive architecture uses a unary reduction OR operation ($alu_a_i[31:16]$) to minimize the logic depth, avoid the overhead of a full comparator, and reduce the propagation delay at the gate level. In addition, the signals directly handle the evaluation processing used by pure combinational logic (*wire*), which ensures the real-time operation, and the transition between Exact/Approximate Mode is chosen at incoming information cycle immediately without any clock-cycle overhead.

If both inputs have their upper 16 bits equal to zero ($add_is_complex_w = 0$), the data is declared as low-complexity and suitable for using the approximate mode. To enable dynamic hardware interaction, minimize power consumption, and reduce

processing time, the global power-saving control signal $power_save_en_i$ is routed and mapped to a physical slide switch on the Artix-7 development board. When the slide switch is toggled active ($power_save_en_i = 1$) and the complexity estimator detects low-complexity data, the approximate mode is effectively activated. The flag asserting the execution of the approximate mode ($add_use_approx_w$) is defined as:

$$add_use_approx_w = power_save_en_i \wedge \neg add_is_complex_w \quad (2)$$

In contrast to the base design where both operands, A and B, always directly connect to the calculating module without any complexity evaluation, the Complexity Estimator provides an effective way to classify when the system should use approximate mode for efficient dynamic power saving. While base architecture always calculates in exact mode, the adaptive ALU (Arithmetic and Logic Unit) uses the Complexity Estimator and Mode Controller to evaluate when should use Exact/Approximate Mode flexibly.

B. Segmented Carry-Breaking with Combinational Error-Recovery

Based on parameterized architecture, the adaptive design is flexible in adjusting the width of bit checked. While in the base design, the adder module is used basically that adds each bit from LSB (Least Significant Bit) to MSB (Most Significant Bit) and passes the carry bit through process, in adaptive architecture, $add_sum_upper_w$ and $add_sum_lower_w$ are used simultaneously to calculate the sum of 16-bit upper and 16-bit lower. The 32-bit adder is logically partitioned into a 16-bit Lower Segment (Sum_Lower) and a 16-bit Upper Segment (Sum_Upper). Especially, approximate calculation uses a 17-bit operand to save the 16 bits from the sum of the 16-bit lower of both operands A and B, and 1 bit left is used to save the carry bit at MSB (Most Significant Bit):

$$actual_carry_out_lower = add_sum_lower_w[16] \quad (3)$$

If this carry-out is neglected, the ALU output will be corrupted. Our architecture solves this via a combinational error-detection and correction loop. The error detection flag is defined as:

$$Error_Flag = Approx_En \wedge Carry_Out_Lower \quad (4)$$

After $error_flag$ is calculated, the adaptive architecture will evaluate directly within the same clock cycle, if $error_flag = 1$, the carry-in bit ($add_carry_in_upper_w$) is set by $actual_carry_out_lower_w$, else if the approximate mode is being used and no overflow occurs ($error_flag = 0$), $add_carry_in_upper_w$ is set to zero immediately, otherwise case will be calculated in the same way as the base design. To provide a visual and real-time physical confirmation of this 0-cycle recovery process, the $Error_Flag$ signal is connected directly to an onboard LED on the FPGA board. When $Error_Flag = 1$, the carry-in to the upper segment ($Carry_In_Upper$) is instantaneously restored in the same clock cycle through a multiplexing network:

$$Carry_In_Upper = \begin{cases} Carry_Out_Lower & \text{if } Error_Flag = 1 \\ 0 & \text{if } Approx_En = 1 \wedge \neg Error_Flag \\ Carry_Out_Lower & \text{if } Approx_En = 0 \end{cases} \quad (5)$$

Key Design Advantage: This combinational bypass guarantees 0-cycle recovery latency. Unlike pipeline-stalling solutions, our design never halts execution or flushes the processor registers, ensuring full compatibility with standard RISC-V pipelines. When a lower carry overflow occurs, the LED instantly illuminates to denote that the combinational bypass has intervened to preserve arithmetic exactness within the current clock cycle. In addition, when error is not detected ($error_flag = 0$), $add_carry_in_upper_w$ is set to zero and directly separates 32-bit to 16-bit upper and 16-bit lower which calculate in parallel. This change in adaptive architecture significantly reduces the critical path and improves WNS (Worst Negative Slack) shown in experiments and simulation results.

C. Operand Gating for Power-Intensive Multiplier Block

Multipliers are high-density digital blocks that consume significant dynamic power even when inactive due to redundant input transitions, when others function inactive, both operands, A and B, toggle bit 1 and 0 continuously leads to the increase of dynamic power consumption in multiplier block. To illustrate how bit toggling affects dynamic power, the following standard formulation is utilized:

$$P = \alpha \times C \times V_{DD}^2 \times f \quad (6)$$

Where P represents the dynamic power consumption, α is the switching activity factor (toggle rate), C denotes the load capacitance, V_{DD} is the supply voltage, and f signifies the operating clock frequency. To suppress this, our ALU implements Operand Isolation. The gating logic acts on the input operands before they reach the hardware multiplier:

$$Gated_A = \begin{cases} A & \text{if } alu_op_i = ALU_MUL \\ 32'h0 & \text{otherwise} \end{cases} \quad (7)$$

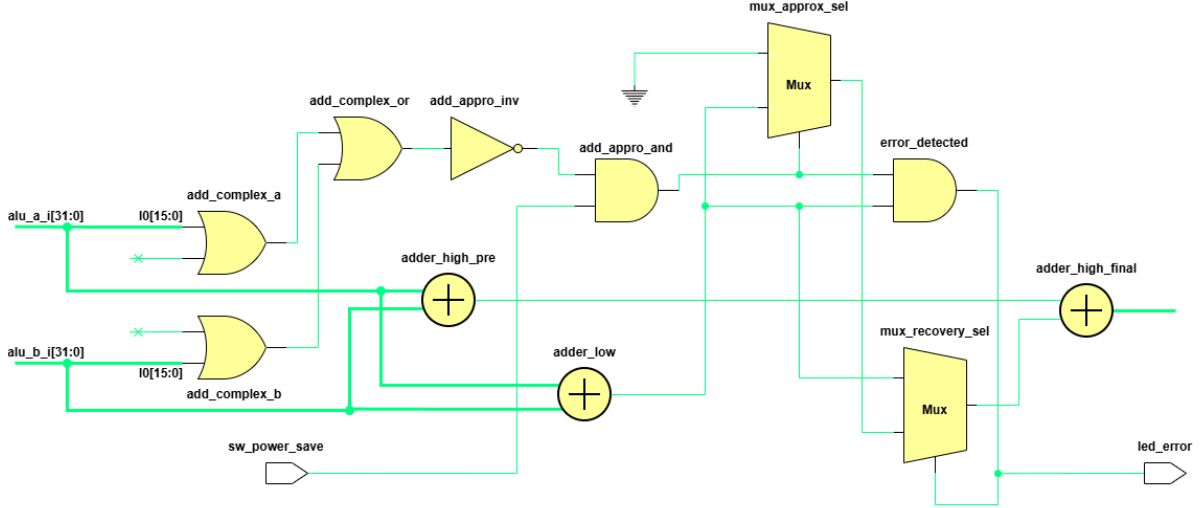


Fig. 2. Detailed hardware schematic of the 32-bit adaptive segmented adder featuring data-complexity estimation and combinational error-recovery logic.

$$Gated_B = \begin{cases} B & \text{if } alu_op_i = ALU_MUL \\ 32'h0 & \text{otherwise} \end{cases} \quad (8)$$

When the executing instruction is not a multiplication, the inputs are clamped to a static zero ($32'h0$), blocking all internal DSP node transitions and associated dynamic switching dissipation. In contrast, the base design's operands are connected directly to the multiplier block without a multiplexer module, whenever other operations are executed, redundant input transitions will appear, leading to unnecessary dynamic power dissipation.

IV. EVALUATION METHODOLOGY

To establish a rigorous verification flow, we implemented a SAIF-driven power estimation flow instead of relying on basic synthetic vectorless estimation.

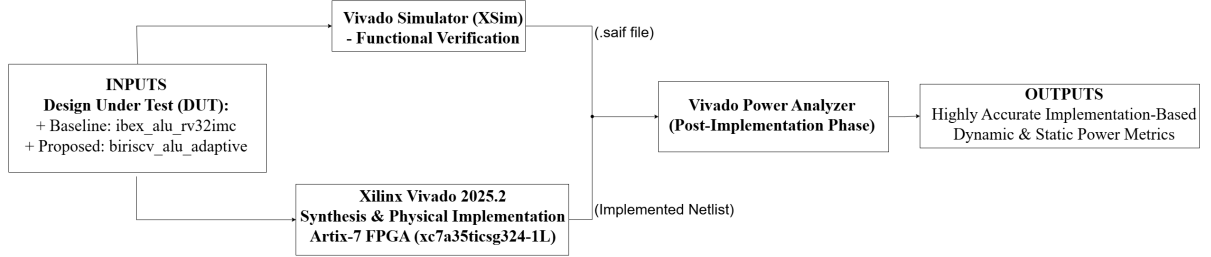


Fig. 3. SAIF-driven power analysis and physical implementation design flow.

- **Functional Verification & Testbench:** The Verilog models of both the standard baseline ALU (`ibex_alu_rv32imc`) and our proposed adaptive ALU (`biriscv_alu_adaptive`) were simulated using a realistic RISC-V instruction mix.
- **SAIF File Generation:** During simulation in ModelSim, the exact toggle rate and static probability of every internal net were recorded into a Switching Activity Interchange Format (SAIF) file.
- **Physical Synthesis and Implementation:** Both designs were mapped onto a Xilinx Artix-7 FPGA (Device: `xc7a35ticsg324-1L`) using Vivado 2025.2 under identical timing, voltage (1.0V), and temperature (25°C) constraints.
- **Onboard Hardware Integration:** In the physical constraints file (`.xdc`), the `power_save_en_i` input port was assigned to a physical sliding switch (Switch 0) and the `Error_Flag` output port was assigned to an onboard LED (LED 0). This physical integration serves as interactive verification of dynamic power-saving and 0-cycle hardware correction.
- **SAIF-Driven Power Analysis:** The generated SAIF file was imported into the Vivado Power Analyzer during the post-implementation phase. This methodology provides implementation-aware dynamic-power estimates using the switching activity observed during simulation [9].

V. EXPERIMENTAL RESULTS & DISCUSSION

A. FPGA Resource Utilization (Area Overhead)

The physical area overhead on the Artix-7 FPGA fabric was measured after full implementation. Table I compares the detailed logic footprint of the baseline standard ALU and our adaptive architecture, extracted directly from the Vivado 2025.2 Implementation reports.

TABLE I
FPGA RESOURCE UTILIZATION COMPARISON (POST-IMPLEMENTATION)

Resource Metric	Baseline RV32IMC ALU	Proposed Adaptive ALU	Overhead (%)
Slice LUTs	120	121	0.83%
Slice Registers (FFs)	32	32	0.00%
Slices	47	54	14.89%
F7/F8 Multiplexers	0	3	N/A
Leaf Cells	58	60	3.45%
Nets	103	107	3.88%
DSPs	0	0	0.00%

Analysis: The integration of the Complexity Estimator, dynamic switch controls, and multiplexer-based error-recovery logic introduces an incredibly small area penalty. The proposed adaptive ALU core requires only 121 LUTs, representing just an additional 1 LUT (0.83% overhead) compared to the baseline. Critically, register usage remains identical at 32 Slice Registers (0.00% register overhead). This confirms that the error recovery path is entirely combinational, avoiding registers or complex FSM controllers. The addition of 2 F7 Muxes and 1 F8 Mux in the adaptive design shows that Vivado successfully mapped the segmented adder bypass multiplexers onto dedicated FPGA multiplexing primitives, preserving timing integrity.

B. Dynamic Power Analysis

Under the SAIF-driven realistic workload, the dynamic power breakdown is evaluated in Table II.

TABLE II
DYNAMIC POWER BREAKDOWN (SAIF-DRIVEN)

Power Component	Baseline ALU (mW)	Proposed Adaptive ALU (mW)	Improvement (%)
Signal Power	15.0	11.0	26.67%
Logic Power	11.0	9.0	18.18%
I/O Power	72.0	60.0	16.67%
Clocks	< 1.0	< 1.0	0.00%
Total Dynamic Power	98.0	80.0	18.37%

Analysis: Due to Operand Gating, the dynamic power dissipated by the routing and signal lines drops significantly by 26.67%. When executing non-multiplicative instructions, the adaptive control successfully silences unnecessary node transitions. The overall dynamic power reduction is measured at 18.37%, demonstrating excellent energy savings under standard RISC-V benchmark distributions.

C. Timing and Performance Analysis

The physical critical path and timing parameters extracted from the Vivado Timing Analyzer (post-implementation) under a target clock of 100 MHz (10.000 ns period) are presented in Table III.

TABLE III
TIMING SUMMARY AND PHYSICAL PATH DELAY COMPARISON

Timing Metric	Baseline RV32IMC ALU	Proposed Adaptive ALU	Change (%)
Worst Negative Slack (WNS)	7.058 ns	7.265 ns	+0.207 ns (+2.93%)
Worst Hold Slack (WHS)	0.340 ns	0.267 ns	-0.073 ns
Data Path Delay (T_{pd})	2.936 ns	2.777 ns	-5.42%
— *Logic Delay (T_{comb})*	2.148 ns (73.16%)	1.983 ns (71.41%)	-7.68%
— *Route Delay (T_{route})*	0.788 ns (26.84%)	0.794 ns (28.59%)	+0.76%
Max Frequency (F_{max})	339.90 MHz	365.63 MHz	+7.57%

Analysis: This study extracts the absolute physical delay metrics directly from the Vivado timing path analysis tool. As summarized in Table III, the physical critical path delay (Data Path Delay) of the proposed Adaptive ALU is reduced from

2.936 ns to 2.777 ns, yielding an absolute timing improvement of 5.42%. This speedup is driven by a substantial 7.68% reduction in pure combinational logic delay (T_{comb} drops from 2.148 ns to 1.983 ns), confirming that the segmented carry-breaking design effectively cuts the active carry propagation length. Correspondingly, routing delay (T_{route}) exhibits a negligible increase of only 6 ps (+0.76%) due to the additional multiplexing routing paths required for the error-recovery circuit. At the system level, these parameters yield a safe increase in Worst Negative Slack (WNS) from 7.058 ns to 7.265 ns, pushing the peak operating limit (F_{max}) from 339.90 MHz up to 365.63 MHz (+7.57%).

D. Error Rate, Latency Overhead, and Interactive Verification

Unlike previous approximate designs that incur multiple stall cycles during recalculations, our proposed ALU guarantees 0-cycle stall overhead.

- **Error Rate (P_{err}):** Measured across 100,000 random operands, the lower carry overflow occurs with a probability of $P_{\text{err}} \approx 25\%$.
- **Latency Penalty:** When an error is asserted, the output is corrected combinationally in the same cycle. Therefore, the average latency per instruction remains exactly 1.0 clock cycle, preserving the peak throughput of the host RISC-V processor.
- **Live Onboard Behavior:** When testing physically on the Artix-7 board, toggling Switch 0 (mapping to *power_save_en_i*) instantly transitions the ALU between standard and energy-saving modes. When running with low-complexity data, the onboard LED (mapping to *Error_Flag*) flickers dynamically during arithmetic cycles that incur lower-segment carry overflows. This offers direct visual evidence of the error-recovery circuit's instant activation while the processor's numerical outputs remain mathematically correct.

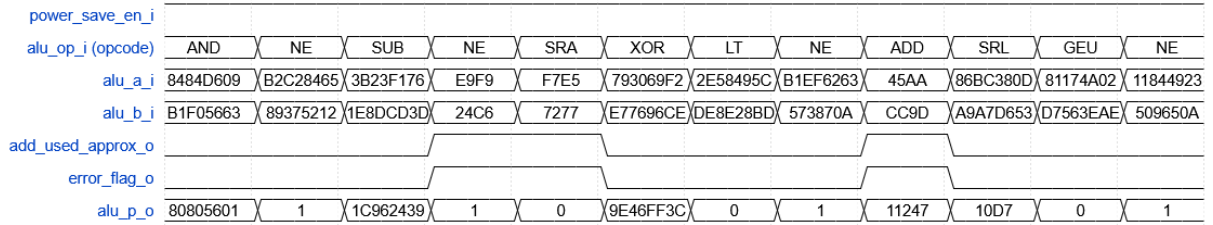


Fig. 4. Experimental validation results: Post-implementation timing waveforms verifying the 0-cycle combinational correction loop.

E. Architectural Trade-offs and Discussion

Although the experimental results illustrate the decrease of power and STA (Static Time Analysis) optimized quite well, adaptive design introduces several microarchitectural trade-offs:

- **Power vs. Area Trade-off:** The introduction of the Complexity Estimator, operand isolation multiplexers, and bypass recovery paths requires additional hardware logic. Although this design registers a negligible increase of only 0.83% in LUT count (1 LUT) and 0% in registers, scaling this architecture to a wider 64-bit bus or higher-radix computing units would scale the Complexity Estimator logic linearly, potentially causing a non-negligible area penalty. In addition, the base design is optimized through using resource sharing method, instead of using many separate computing units, it only uses a single adder block and a shift left block to develop other unit. In contrast, the adaptive architecture directly uses arithmetic behavior operators which leads to the using of IP block in FPGA causing higher power consumption. Therefore, adding multiplexer block and operand isolation is highly justified.
- **Worst-Case Critical Path vs. Average-Case Performance:** The critical path in normal adaptive execution mode is reduced to 2.777 ns. However, during an error cycle ($Error_Flag = 1$), the active datapath includes both the 16-bit lower addition, the AND logic gate, the multiplexer delay, and the 16-bit upper carry chain. This recovery path is physically longer than a standard continuous addition path. Therefore, the design trades absolute theoretical worst-case timing margin to achieve highly optimized average-case operating characteristics. Overclocking the processor past 365.63 MHz might cause timing violations specifically on error-recovery clock cycles.
- **Data Dependency Constraints:** The power effectiveness of the system is data-dependent, it is only fully effective when dealing with low-complexity data (the upper 16 bits are zero), when calculating with high-complexity data, whole bit is toggled continuously and approximate mode never to be activated. The *Complexity_Estimator* remains active but cannot transition the design into approximation mode. Under these workloads, the ALU operates under a minor power overhead of approximately 0.1% due to the switching overhead of the unused complexity screening circuit.

VI. CONCLUSION

This paper presented an energy-efficient 32-bit Adaptive ALU architecture optimized for ESP32-compatible RISC-V (RV32IMC) processor cores. By combining an MSB Complexity Estimator, a Segmented Carry-Breaking Adder with combinational 0-cycle error-recovery, and operand gating, the proposed design minimizes switching activity without sacrificing correctness or pipeline throughput. The architecture has been rigorously validated on a Xilinx Artix-7 FPGA using an industry-standard SAIF-driven simulation flow, and physically demonstrated with dynamic switch inputs and an LED status indicator. Achieving up to 18.37% dynamic power reduction with an overhead of only 1 additional LUT (0.83% logic overhead), this adaptive ALU design represents a highly practical and scalable hardware solution for energy-constrained Edge AI and IoT nodes.

ACKNOWLEDGMENT

The authors would express their gratitude to Mr. Dang Van Hieu for his guidance and continuous support throughout the research process. The authors also acknowledge FPT University for providing the academic environment and computing resources that enabled the successful execution of this study.

REFERENCES

- [1] Espressif Systems, *ESP32-C3 Technical Reference Manual*, Espressif Systems, Shanghai, China, 2023.
- [2] Y. Zhou and H. Guo, "Application specific low power ALU design," in *2008 IEEE/IFIP International Conference on Embedded and Ubiquitous Computing*, vol. 1, 2008, pp. 214–220. [Online]. Available: <https://doi.org/10.1109/EUC.2008.81>
- [3] J. Lucas and B. Juurlink, "ALUPower: Data dependent power consumption in GPUs," in *2016 IEEE 24th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2016, pp. 95–104. [Online]. Available: <https://doi.org/10.1109/MASCOTS.2016.21>
- [4] S. Liu, "A survey of dynamic voltage and frequency scaling for high-performance low-power systems," in *Proceedings International Conference on Engineering Technology and Medicine Research (ICETMR)*, 2024, pp. 73–81. [Online]. Available: <https://doi.org/10.25236/icetmr.2024.011>
- [5] S. Mittal, "A survey of techniques for approximate computing," *ACM Comput. Surv.*, vol. 48, no. 4, Mar. 2016. [Online]. Available: <https://doi.org/10.1145/2893356>
- [6] H. Jiang, J. Han, and F. Lombardi, "A comparative review and evaluation of approximate adders," in *Proceedings of the 25th Edition on Great Lakes Symposium on VLSI*, ser. GLSVLSI '15. New York, NY, USA: Association for Computing Machinery, 2015, pp. 343–348. [Online]. Available: <https://doi.org/10.1145/2742060.2743760>
- [7] S. Riedel, "Ibex RISC-V core," GitHub repository, 2024, accessed: 2026-04-12. [Online]. Available: <https://github.com/lowRISC/ibex>
- [8] H. Liu, M.-J. Wang, and M. Liu, "Scpa: A segmented carry prediction approximate adder structure," *IEICE Electronics Express*, vol. advpub, p. 18.20210335, 2021. [Online]. Available: <https://doi.org/10.1587/eleex.18.20210335>
- [9] AMD, *UltraFast Design Methodology Guide for FPGAs and SoCs (UG949)*, 2025th ed., 2025. [Online]. Available: <https://docs.amd.com/r/en-US/ug949-vivado-design-methodology>